SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Group Name: Cemicolon

Group Members: Cadee, Evelyn, Madeline

Project Name: LexiFlow AI

Domain: 1

1. Introduction

1.1 Purpose

This System Analysis Documentation (SAD) describes the technical scope, system architecture, design decisions, data flow, AI reasoning process, and implementation strategy of LexiFlow AI.

LexiFlow AI is an AI-powered compliance monitoring system that helps users detect risky personal data sharing in real time, especially when writing emails or working with online documents.

This document covers:

1. System architecture
2. Data flow
3. GLM reasoning process
4. API design
5. Functional and non-functional requirements
6. Monitoring, assumptions, dependencies, and future improvements

This document acts as a reference for developers, testers, and project stakeholders to understand how LexiFlow AI operates from frontend detection to backend AI reasoning and audit logging.

1.2 Background

Users often unintentionally share sensitive information such as names, phone numbers, emails, bank account numbers, or customer data through emails and online platforms.

Current writing platforms such as Gmail and Google Drive do not provide real-time compliance reasoning before users send or share sensitive content. As a result, users may accidentally expose personal data without proper consent, security control, or review.

LexiFlow AI introduces a real-time compliance assistant that:

- Scans user-generated content
- Detects risky personal data
- Calls ILMU-GLM-5.1 for AI reasoning
- Returns risk level, explanation, and safe writing suggestions
- Stores audit logs for dashboard review and report generation

The system is designed as a browser extension and dashboard supported by a FastAPI backend and GLM service layer.

1.3 Target Stakeholders

Stakeholder	Role	Expectations
End Users	Write emails or online content	Receive real-time warning before sending risky content
Compliance Reviewers	Review audit logs and reports	Understand what risks were detected and why
Product / Project Lead	Define product vision and scope	Ensure MVP meets hackathon goals
Frontend Developer	Build extension and dashboard	Clear API schema and stable backend endpoints
Backend / AI Developer	Build APIs and GLM integration	Reliable backend, structured JSON output, and fallback logic
QA Tester	Validate system behaviour	Defined test cases, predictable API responses, and edge case coverage

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

Type	Details
System	Browser Extension + Web Dashboard
Architecture	Client-server architecture with GLM as AI service layer
Backend	FastAPI deployed on Render
AI Model	ILMU-GLM-5.1
Storage	In-memory audit log for MVP
Frontend	Gmail/Drive browser extension and dashboard UI

LexiFlow AI is structured as a client-server system. The frontend includes a browser extension that monitors user input in Gmail and supported web pages, and a dashboard that displays audit logs and reports. The frontend sends content to the FastAPI backend through REST API calls.

The backend processes the input, constructs a compliance reasoning prompt, sends the prompt to ILMU-GLM-5.1, parses the structured model response, returns the result to the frontend, and stores the latest audit record.

2.1.2 LLM as Service Layer

LexiFlow AI integrates ILMU-GLM-5.1 as the main reasoning engine, not just as a generic AI module.

The GLM service layer is responsible for:

1. Understanding the user's input content
2. Detecting personal or sensitive information
3. Evaluating whether the content may violate privacy or compliance expectations
4. Producing structured JSON output
5. Generating explanations and safe writing suggestions

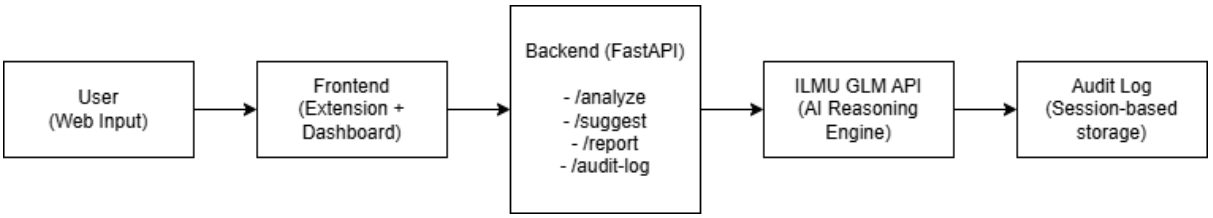
The backend constructs a prompt containing:

- Source type, such as email_page, web_page, uploaded_pdf, or uploaded_image
- Trigger mode, such as auto_scan or send_attempt
- User text content

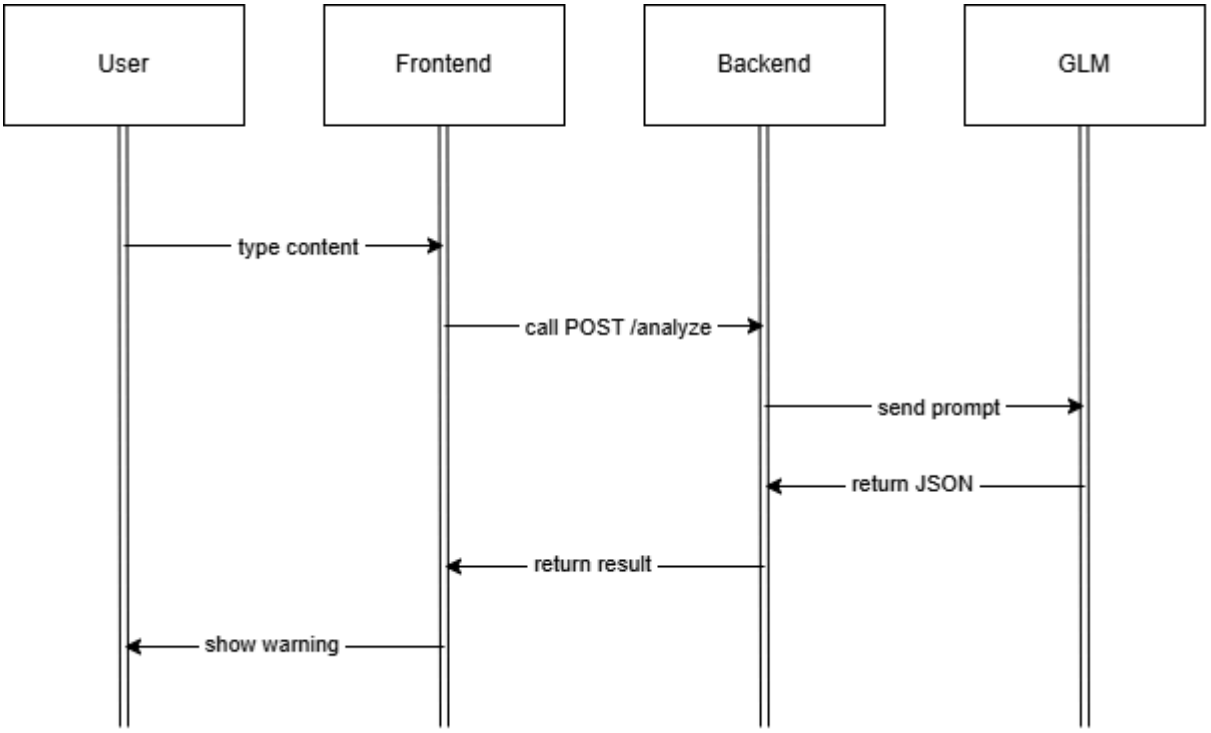
- Required JSON response fields
- Compliance reasoning rules

The GLM output is parsed and normalized by the backend before being sent to the frontend. If GLM fails, the system falls back to rule-based logic to maintain reliability.

2.1.3 Dependency Diagram



2.2 Sequence Diagram



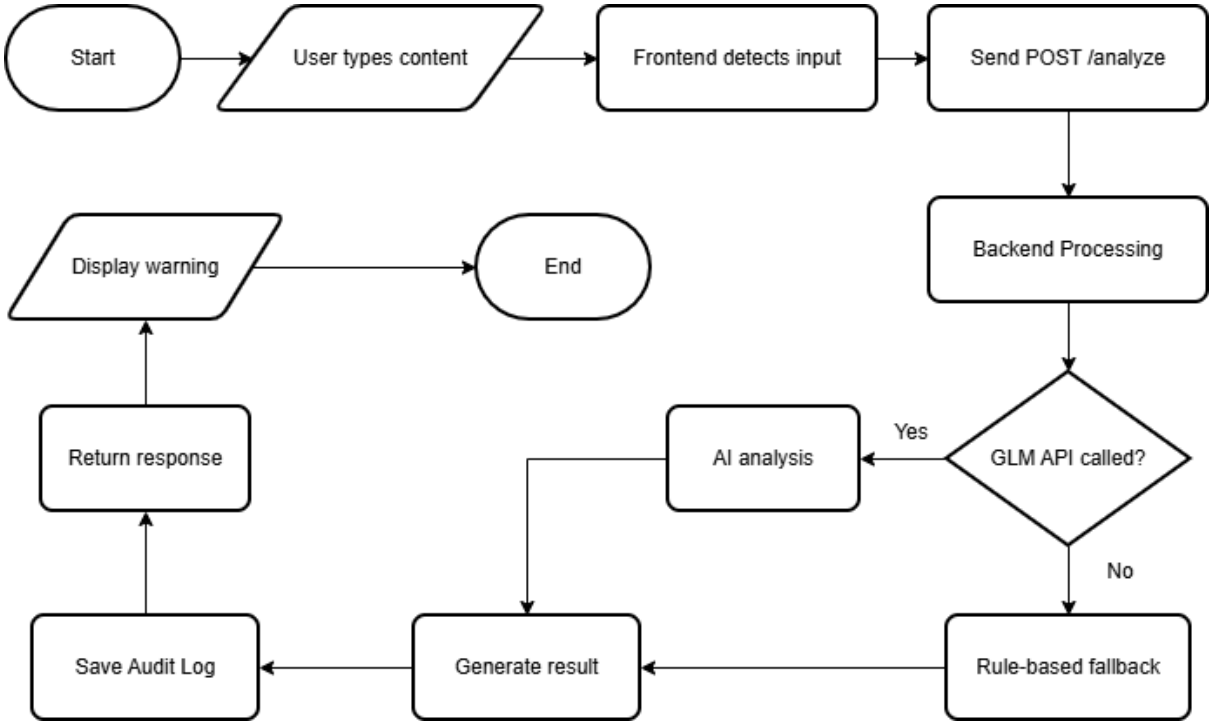
2.3 Technological Stack

Component	Technology	Purpose
Frontend Extension	JavaScript / Chrome Extension	Monitor Gmail and supported web pages
Dashboard	Next.js / React	Display audit logs, reports, and settings

Component	Technology	Purpose
Backend	FastAPI	Provide API endpoints and process requests
AI Reasoning Engine	ILMU-GLM-5.1	Generate compliance reasoning and structured output
Deployment	Render	Host backend with trusted HTTPS endpoint
Version Control	GitHub	Code collaboration and submission
Storage	In-memory list	MVP audit log storage

2.4 Key Data Flows

2.4.1 Data Flow Diagram



2.4.2 Data Structure

Analyze Request

```
{
  "input": "string",
  "source_type": "email_page",
  "trigger_mode": "send_attempt",
  "session_id": "unique_session_id"
}
```

Analyze Response

{
  "status": "approve | blocked | escalate",
  "risk_level": "low | medium | high",
  "risk_score": 85,
  "reason": "string",
  "suggestion": "string",
  "detected_entities": ["string"],
  "compliance_basis": "string",
  "reasoning_steps": ["string"],
  "recommended_action": "allow | show_warning",
  "safe_email_suggestion": "string",
  "source_type": "email_page",
  "trigger_mode": "send_attempt",
  "session_id": "unique_session_id",
  "timestamp": "YYYY-MM-DD HH:MM:SS"
}

Audit Log Record

{
  "input": "string",
  "source_type": "email_page",
  "trigger_mode": "send_attempt",
  "session_id": "unique_session_id",
  "status": "blocked",
  "risk_level": "high",
  "risk_score": 85,
  "recommended_action": "show_warning",
  "timestamp": "YYYY-MM-DD HH:MM:SS"
}

2.5 API Design

Endpoint	Method	Purpose
/	GET	Check backend running status
/health	GET	Check GLM readiness
/analyze	POST	Analyze content and return compliance result
/suggest	POST	Generate safer rewritten content
/audit-log	GET	Return audit history

Endpoint	Method	Purpose
/report	GET	Generate audit report summary
/risk-score	GET	Placeholder endpoint for future risk score expansion

3. Functional Requirements & Scope

3.1 Minimum Viable Product

#	Feature	Description
1	Real-time Content Analysis	Extension monitors user input and sends text to backend
2	AI Compliance Reasoning	Backend uses ILMU-GLM-5.1 to classify risk
3	Risk Decision Output	System returns approve, blocked, or escalate
4	Safe Writing Suggestion	System suggests safer email wording
5	Audit Log Dashboard	Dashboard displays scan history
6	Report Generation	Dashboard generates summary report by period or date range

3.2 Non-Functional Requirements

Quality	Requirement	Implementation
Performance	API should respond quickly for real-time scanning	Lightweight FastAPI backend and short prompt structure
Reliability	System should continue working if GLM fails	Rule-based fallback mechanism
Maintainability	Code should be easy to update	Separated frontend and backend folders
Scalability	System should support more platforms in future	Generic source_type and trigger_mode request fields
Security	API key must not be hardcoded	Environment variables in local machine and Render
Token Latency	GLM response should be suitable for demo flow	Fixed prompt format and short input payload
Cost Efficiency	Avoid unnecessary model calls	Frontend uses session-based scanning and backend stores latest session result

3.3 Out of Scope / Future Enhancements

The following features are planned for future versions but are not fully implemented in the MVP:

- Persistent database for audit logs
- Full PDF text extraction
- Image OCR scanning
- Spreadsheet text extraction
- Multi-platform extension support beyond Gmail and Google Drive
- Role-based user authentication
- Advanced compliance policy customization
- Export report as PDF

4. Monitoring, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

The system is evaluated using the following test cases:

Test Case	Expected Result
Email contains no personal data	Approve / low risk
Email contains personal data only	Approve or escalate depending on context
Email contains bank account number	High risk / warning
Email shares customer data externally without consent	Blocked / high risk
User removes sensitive data from same email session	Audit log updates to latest lower-risk result
GLM API unavailable	Rule-based fallback still returns response

4.2 Monitoring Plan

Monitoring is performed through:

- /health endpoint to check backend and GLM readiness
- Render deployment logs
- Frontend error messages
- Audit log records
- Report summary counts

If the GLM API fails, the backend uses rule-based fallback and still returns a valid structured response.

4.3 Priority Matrix

Priority	Trigger Condition	Action
P1 Critical	Backend unavailable or /analyze fails	Use Render logs, redeploy backend, verify /health

Priority	Trigger Condition	Action
P2 High	GLM API fails or quota unavailable	Use rule-based fallback
P3 Medium	Audit log duplicate issue	Use session_id to update existing record
P4 Low	Report filter issue	Adjust query parameters and filter logic

4.4 Assumptions

- Users have a stable internet connection
- Browser extension has permission to read visible page content
- Frontend sends extracted text to backend
- GLM API key is valid and stored securely
- Audit log storage is temporary for MVP
- Users understand that the system provides compliance assistance, not final legal advice

4.5 External Dependencies

Tool	Purpose	Risk	Mitigation
ILMU-GLM-5.1 API	Core AI reasoning engine	API rate limit, latency, or failure	Rule-based fallback
Render	Backend deployment	Cold start or service downtime	Warm up backend before demo
GitHub	Version control	Merge conflicts	Use main branch as final branch
Chrome Extension	Browser integration	Permission or platform limitations	Limit MVP to Gmail and supported visible web pages
FastAPI	Backend API framework	Dependency/version issue	Keep requirements updated

5. Project Management & Team Contributions

5.1 Project Timeline

Date	Work Completed
18 Apr	Project scope, backend setup, initial repository
19 Apr	/analyze endpoint and mock rule-based logic
20 Apr	Risk score and structured JSON response
21 Apr	/audit-log endpoint and frontend integration
22 Apr	Extension/dashboard integration and diagrams
23 Apr	ILMU-GLM integration and Render deployment
24 Apr	Report generation, safe suggestion, final testing
25 Apr	Final documentation, pitch preparation, submission cleanup

5.2 Team Members Role

Member	Role	Contribution
Cadee	Product / PM	PRD, pitch deck, storyline, scope control
Evelyn	Frontend / QA	Gmail extension, dashboard, testing, demo environment
Madeline	Backend / AI / Architecture	FastAPI backend, GLM integration, API schema, SAD, deployment

5.3 Recommendations

For future scalability and reliability, the following improvements are recommended:

1. Replace in-memory audit log with persistent database such as PostgreSQL or MongoDB
2. Add authentication for compliance dashboard users
3. Add PDF/OCR/spreadsheet extraction for document scanning
4. Add caching or debouncing to reduce unnecessary GLM calls
5. Add exportable PDF compliance reports

6. Expand browser extension support to Outlook, WhatsApp Web, and other writing platforms

6. Conclusion

LexiFlow AI demonstrates a working AI-powered compliance monitoring workflow using a browser extension, FastAPI backend, ILMU-GLM-5.1 reasoning engine, audit log dashboard, report generation, and safe writing suggestions.

The system successfully shows how AI can assist users in detecting sensitive data risks before sending or sharing content. Its modular architecture allows future expansion into more platforms, persistent storage, and advanced compliance policies.